Autómata del código secreto

Caso práctico

Carlos tiene que programar el juego del código secreto (TAREA-02 AVANZADA). Antes de escribir una línea, **María** le pide el **diagrama de estado-transición ampliado** y las tablas del autómata. — Así sabrás qué páginas hay, qué botón se puede pulsar en cada una y qué hay que comprobar antes de cambiar de página. Y lo que es más importante en un juego: **dónde no te puedes fiar** de lo que manda el navegador.

Este documento aplica el método de los diagramas de estado-transición ampliados a la tarea **TAREA-02-AVANZADA-CodigoSecreto**. La solución (`../SOLUCION-01/index.php`) está programada **a partir de este autómata**: las constantes `ST0`…`ST4` y `EE0`…`EE6`, el array `$automata` y un `case` por transición.

1.- Cómo se lee el diagrama

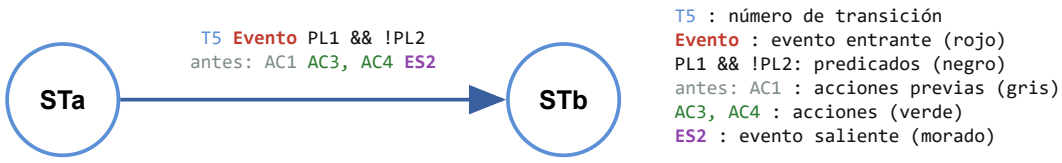


Figura 1. Leyenda del diagrama de estado-transición ampliado

2.- El problema, en resumen

- Se configura la partida: **posiciones** (4 o 5), **colores** (6, 7 u 8), **intentos** (6 a 12) y si los colores se pueden **repetir**.
- El servidor elige un **código secreto** que **no se envía a la página**: se calcula cada vez a partir de una semilla con `hash_hmac()` y una clave que solo conoce el servidor.
- En cada intento se eligen los colores; el servidor contesta con las **negras** (color y posición bien) y las **blancas** (color bien, posición mal).
- Una **pista** por partida enseña el color de una posición y **cuesta un intento**. El jugador también se puede **rendir**.
- La partida termina **ganada** (acierta el código) o **perdida** (sin intentos o rendido); el **marcador** se conserva entre partidas.

3.- Análisis

- **Eventos entrantes.** La carga sin parámetros y un botón por acción: *Empezar*, *Probar*, *Pista*, *Rendirse*, *Nueva partida* y *Cambiar la configuración*. Todos llegan en el campo `evento` (`<button name="evento" value="probar">`).
- **Estados.** Cuatro situaciones en las que la página espera algo distinto, más ST0: la configuración, el juego, la partida ganada y la partida perdida. Ganada y perdida admiten los mismos eventos, pero la

página es distinta y el marcador también: son dos estados.

- **Variables de estado.** La configuración, la **semilla**, los intentos hechos (`"0123-4512"`), la pista y el marcador. El código secreto **no** es una variable de estado: es una **acción** (AC4) que se repite en cada petición. Por eso se puede ver el código fuente de la página sin hacer trampas.
- **Dónde está la desconfianza.** En ST2 llegan la configuración, la semilla y los intentos en campos ocultos. PL1 comprueba que son una partida **posible en ese estado**: configuración válida, intentos bien formados, ninguno acertado (si no, la partida ya habría terminado) y que quedan intentos.
- **Un predicado que se evalúa después de una acción.** PL4 ("era el último intento") se evalúa **después** de añadir el intento (AC6). AC6 se hace en todos los caminos con PL2, así que no cambia el resultado de los otros predicados.

4.- Diagrama de estado-transición ampliado

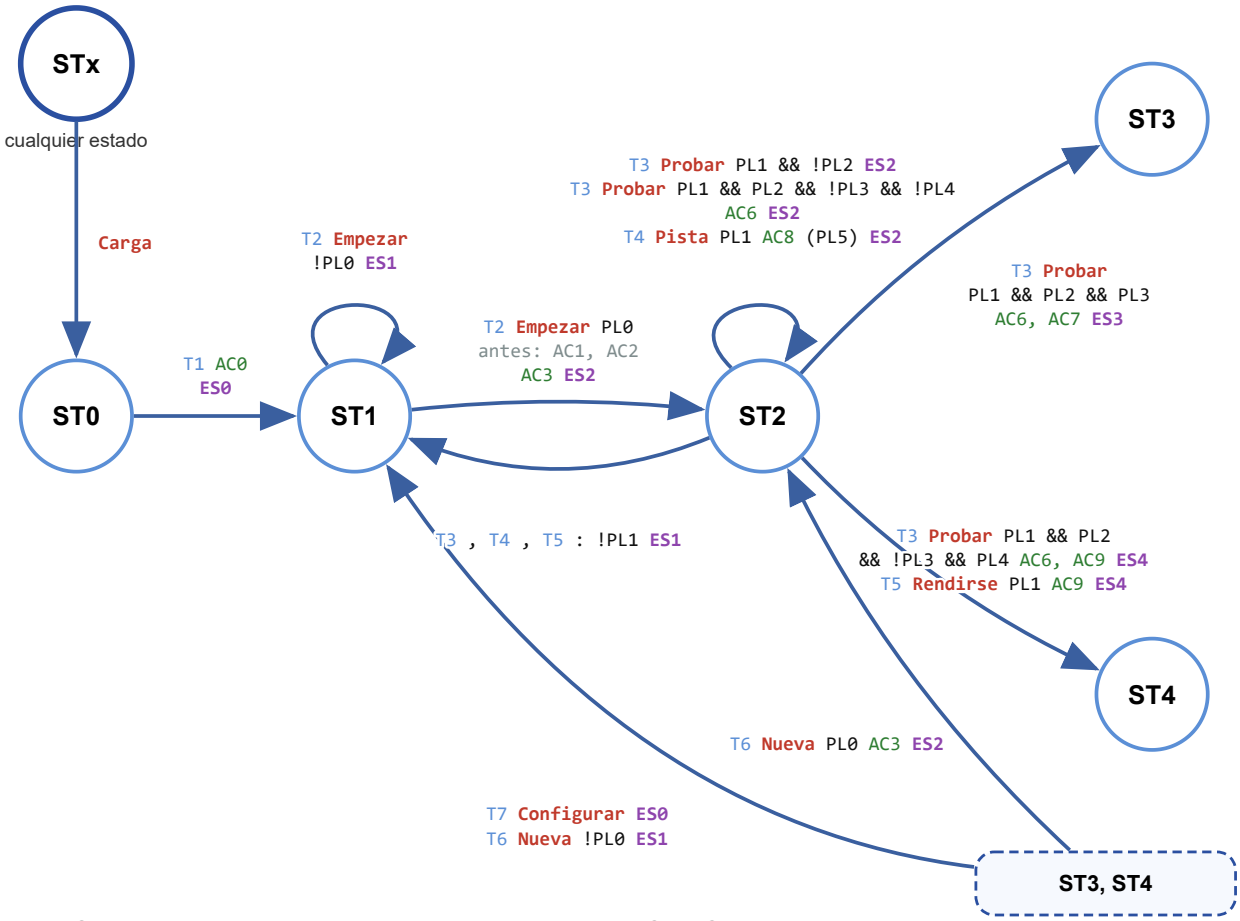


Figura 2. Diagrama del código secreto (la caja discontinua vale para ST3 y ST4)

5.- Autómata

5.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Carga	petición sin `evento` (primera vez o recarga)
Evento entrante	EE1	Empezar	enviar la configuración
Evento entrante	EE2	Probar	enviar un intento (`intento[0]`, `intento[1]`...)
Evento entrante	EE3	Pista	pedir la pista
Evento entrante	EE4	Rendirse	
Evento entrante	EE5	Nueva	otra partida con la misma configuración
Evento entrante	EE6	Configurar	"Cambiar la configuración"
Evento saliente	ES0 / ES1	configuración	formulario y marcador; ES1 con los errores o un aviso
Evento saliente	ES2	juego	intentos anteriores con negras y blancas, pista, formulario del intento (con aviso opcional)
Evento saliente	ES3 / ES4	ganada / perdida	historial, código descubierto, marcador, *Nueva* y *Configurar*
Estado	ST0	Carga	la petición llega sin estado
Estado	ST1	Configuración	
Estado	ST2	Jugando	
Estado	ST3 / ST4	Ganada / Perdida	
Variable de estado	VE0	`$estado`	campo `estado`
Variable de estado	VE1	configuración	`posiciones`, `colores`, `maxintentos`, `repite`
Variable de estado	VE2	`semilla`	de ella sale el código secreto (con la clave del servidor)
Variable de estado	VE3	`intentos`	`"0123-4512"`: un grupo de cifras por intento
Variable de estado	VE4	`pista`	posición revelada (-1 si no se ha pedido)
Variable de estado	VE5	marcador	`ganadas`, `perdidas`
Predicado	PL0	configuración válida	posiciones 4 o 5, colores 6, 7 u 8, intentos de 6 a 12

Tipo	Abreviatura	Nombre	Significado
Predicado	PL1	partida posible	configuración y semilla válidas, intentos bien formados, ninguno acertado y quedan intentos
Predicado	PL2	intento válido	un color válido en cada posición y sin repetir si la partida no lo permite
Predicado	PL3	acierta	el intento es el código secreto
Predicado	PL4	último intento	después de añadirlo, no quedan intentos
Predicado	PL5	se puede pedir la pista	no se ha pedido y quedan 2 intentos o más
Acción	AC0	empezar de cero	configuración por defecto y marcador a cero
Acción	AC1	recuperar	la partida o el marcador de los campos ocultos
Acción	AC2	leer la configuración	del formulario
Acción	AC3	nueva partida	semilla nueva, sin intentos ni pista
Acción	AC4	calcular el código secreto	`hash_hmac()` de la semilla con la clave, `mt_srand()` y `shuffle()`
Acción	AC5	leer el intento	
Acción	AC6	añadir el intento	
Acción	AC7 / AC9	ganadas++ / perdidas++	
Acción	AC8	elegir la pista	la primera posición que no se ha acertado nunca
Acción	AC10	guardar	el estado y la partida en campos ocultos (en ST1, solo el marcador)

5.2.- Tabla de eventos-estados ampliada

	Carga	Empezar	Probar	Pista	Rendirse	Nueva	Configurar
ST0	T1	T1	T1	T1	T1	T1	T1
ST1	T1	T2	---	---	---	---	---
ST2	T1	---	T3	T4	T5	---	---
ST3	T1	---	---	---	---	T6	T7
ST4	T1	---	---	---	---	T6	T7

Las celdas `---` no pueden darse con la página: no hay botón para ese evento en ese estado. Si llegan, se trata como T1.

5.3.- Tabla de transiciones

#	AEs antes	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	AC0	ES0 [AC10]
T2	AC1, AC2	`!PL0`	ST1	---	ES1 [AC10]
T2	AC1, AC2	`PL0`	ST2	AC3	ES2 [AC10]
T3	AC1, AC4, AC5	`!PL1`	ST1	AC0	ES1 "partida no válida" [AC10]
T3	AC1, AC4, AC5	`PL1 && !PL2`	ST2	---	ES2 con el motivo [AC10]
T3	AC1, AC4, AC5, AC6	`PL1 && PL2 && PL3`	ST3	AC7	ES3 [AC10]
T3	AC1, AC4, AC5, AC6	`PL1 && PL2 && !PL3 && PL4`	ST4	AC9	ES4 [AC10]
T3	AC1, AC4, AC5, AC6	`PL1 && PL2 && !PL3 && !PL4`	ST2	---	ES2 "n negras y m blancas" [AC10]
T4	AC1, AC4	`!PL1`	ST1	AC0	ES1 [AC10]
T4	AC1, AC4	`PL1 && !PL5`	ST2	---	ES2 con el motivo [AC10]
T4	AC1, AC4	`PL1 && PL5`	ST2	AC8	ES2 [AC10]
T5	AC1, AC4	`!PL1`	ST1	AC0	ES1 [AC10]
T5	AC1, AC4	`PL1`	ST4	AC9	ES4 [AC10]
T6	AC1	`!PL0`	ST1	AC0	ES1 [AC10]
T6	AC1	`PL0`	ST2	AC3	ES2 [AC10]
T7	AC1	---	ST1	---	ES0 [AC10]

5.4.- Comprobación de predicados

T3 (con **x** = no se evalúa: va detrás en el `&&` o en un `if` anterior):

PL1	PL2	PL3	PL4	no válida	intento mal	gana	pierde	sigue
0	x	x	x	1	0	0	0	0
1	0	x	x	0	1	0	0	0
1	1	1	x	0	0	1	0	0
1	1	0	1	0	0	0	1	0
1	1	0	0	0	0	0	0	1

En cada fila hay exactamente un 1: los caminos son **completos** y **excluyentes**. Si el último intento es además el que acierta (PL3 y PL4), **gana**: por eso PL3 se comprueba antes que PL4.

T4:

PL1	PL5	no válida	sin pista	pista
0	x	1	0	0
1	0	0	1	0
1	1	0	0	1

6.- Del autómata al código

```
//               Carga Empezar Probar Pista Rendirse Nueva Configurar
$automata = [
    ST0 => [1, 1, 1, 1, 1, 1, 1],
    ST1 => [1, 2, null, null, null, null, null],
    ST2 => [1, null, 3, 4, 5, null, null],
    ST3 => [1, null, null, null, null, 6, 7],
    ST4 => [1, null, null, null, null, 6, 7],
];
switch ($automata[$estado][$evento] ?? 1) {   // null ("---") se trata como T1
    case 3:                                   // ---- T3: ST2 + Probar
        $partida = leerPartida($_POST);       // AC1
        $pl1 = partidaValida($partida);       // PL1
        ...
        $secreto = codigoSecreto($partida);   // AC4
        [$intento, $motivo] = leerIntento($_POST, $partida);   // AC5
        $pl2 = $intento !== null;             // PL2
```

Tres decisiones que salen del autómata:

- **El código secreto es una acción, no una variable de estado.** `codigoSecreto()` (AC4) hace `mt_srand(crc32(hash_hmac('sha256', semilla, CLAVE)))` y baraja: con la misma semilla da siempre el mismo código, y sin la clave no se puede calcular.
- **En ST1 solo se guarda el marcador.** El formulario de configuración tiene campos `posiciones`, `colores`... Si AC10 escribiera también campos ocultos con esos nombres, el navegador mandaría los dos y PHP se quedaría con el **último** (el oculto): la configuración elegida no serviría de nada.
- **`default` junto a `case 1`.** Durante la construcción paso a paso, una transición de la tabla que todavía no está programada se comporta como T1 en lugar de dejar la página en blanco.

7.- Pruebas: un camino por fila

Las 51 peticiones de `pruebas.json` (en `fuentes-pasos`) recorren todas las filas de la tabla de transiciones. Con la semilla 1, 4 posiciones, 6 colores y sin repetir, el código es **rojo, morado, verde, naranja**:

Camino	Petición	Resultado
T1	primera carga; `probar` en ST1; `estado=7`	configuración con el marcador a cero
T2 `!PL0`	posiciones 6, colores 9, intentos 20	tres errores, sigue en ST1
T2 `PL0`	4, 6, 10, semilla 1	ST2, 10 intentos
T3 `PL1 && !PL2`	una posición sin color; dos colores iguales	aviso, sigue en ST2
T3 `... && !PL4`	rojo, verde, azul, amarillo	"1 negras y 1 blancas.", quedan 9
T4 `PL1 && PL5`	Pista	"en la posición 2 va morado", quedan 8
T4 `PL1 && !PL5`	Pista otra vez; pista con un solo intento	"Ya has usado la pista"; "No te quedan intentos"
T3 `... && PL3`	rojo, morado, verde, naranja	"¡Has descubierto el código en 2 intentos!"
T6 `PL0`, T5 `PL1`, T7	Nueva, Rendirse, Configurar	marcador 1 ganada, 2 perdidas
T3 `... && PL4`	6 intentos fallados	"Se han acabado los intentos."
T3/T4 `!PL1`	`intentos=0514` (ya acertado), `pista=7`, `semilla=0`, 10 intentos de 10...	"La partida recibida no es válida"

8.- Para practicar

- Añade un evento *Deshacer* que quite el último intento (solo si no se ha pedido la pista). ¿En qué estados aparece? ¿Qué predicados necesita?
- Cambia la pista para que **no** cueste un intento pero solo se pueda pedir a partir del tercer intento. ¿Qué filas de la tabla de transiciones cambian?
- Añade un estado *Pausa* (con un botón *Pausar* y otro *Seguir*). ¿Cambia algo en PL1?
- Haz la tabla de verdad de T6 y comprueba que es completa y excluyente.

Autoevaluación

- ¿Por qué el código secreto no es una variable de estado? **Porque viajaría en la página y se podría leer con el código fuente. Se calcula en cada petición (AC4) a partir de la semilla y de una clave que solo tiene el servidor.**
- ¿Por qué ganada y perdida son dos estados si admiten los mismos eventos? **Porque la página que se muestra es distinta (ES3 y ES4) y lo que se ha sumado al marcador también. Un estado es una situación de la página, no solo los eventos que admite.**
- ¿Qué pasa si en ST2 llegan los intentos `"0514"` con la semilla 1? **Ese intento es el código secreto: la partida ya habría terminado. PL1 es falso y se vuelve a la configuración (T3, primer camino).**
- En T3, ¿por qué PL3 se comprueba antes que PL4? **Porque si el último intento acierta, la partida se gana: con el orden contrario, se perdería una partida ganada.**